Introduction **1**
Task Description **2**
Data 2
State of the Art 2
pjmathematician at the CLEF 2025 JOKER Lab Tasks 1, 2 & 3: A Unified Approach to Humour Retrieval and Translation using the Qwen LLM Family 2
Enhancing Humor-Aware Information Retrieval with Relevance-Aware Classification 3
Evaluation of multiple models for pun detection and where they fail 6
Exploratory Data Analysis (EDA) **9**
First Implementation Attempt 15
Semantic Retrieval Stage 15
Construction of Training Pairs 16
Pun Detection Training 16
Inference and Ranking 16
Experimental results 17
Conclusions 18
Second implementation **19**
Architecture 19
Experimental results 19
Conclusion 21
Prompt Engineering **22**
Model 22
Prompts 22
Experimental results 24
Moving forward 25
Second Prompt Engineering **25**
Offline Humor Analysis 26
Corpus Filtering and Text Preparation 26
Dense Retrieval 26
Results 27
Conclusion 27
Bibliography 27

Introduction

Humor and wordplay constitute non-literal forms of language that remain difficult for contemporary artificial intelligence systems to process. Their interpretation often depends on implicit cultural references, multiple concurrent meanings, and atypical interactions between linguistic form and semantic content. Additionally, many types of wordplay rely on surface-level linguistic features (such as orthography, pronunciation, or other form-based cues) which are not explicitly modeled by current NLP architectures. As a result, tasks

involving humor detection, pun identification, or humor-aware retrieval remain challenging for state-of-the-art systems.

The JOKER Lab focuses precisely on these aspects by developing reusable test collections and tasks dedicated to AI's understanding and handling of humor and wordplay.

Task Description

In the CLEF 2026 JOKER Lab, Task 1 focuses on humor-aware information retrieval. The objective is to retrieve short humorous texts from a large document collection in response to a given query. Retrieved documents must satisfy two criteria: they must be relevant to the topic expressed by the query and they must contain wordplay.

For example, given the query "math", the system is expected to retrieve jokes related to mathematics, such as texts involving mathematical concepts or puns (e.g., jokes based on the idea of finding common denominators).

Data

The dataset provided for this task corresponds to the 2025 release and includes document collections in both English and Portuguese. Each collection contains short texts, some of which include wordplay, while others are non-humorous.

For each language, the dataset is divided into four JSON files, which we refer to using abbreviated names for clarity:

- **`corpus:`** contains the full set of documents. Each entry includes a unique *docid* and a short *text*.
- **`queries_train`**: provides the training queries, each defined by a *qid* and a *query* term indicating the topic for which relevant humorous texts should be retrieved.
- **`queries_test`**: has the same structure as the training query file, but is used exclusively for evaluation
- **`qrels_train`**: contains the relevance judgments for the training queries. Each record links a *qid* to a *docid*, with *qrel = 1* indicating that the document is both topically relevant and a wordplay instance, and *qrel = 0* otherwise.

State of the Art

pjmathematician at the CLEF 2025 JOKER Lab Tasks 1, 2 & 3: A Unified Approach to Humour Retrieval and Translation using the Qwen LLM Family

Given that Task 1 requires retrieving texts that are both topically relevant and humorous, previous editions of the JOKER Lab provide the most directly applicable technological landscape. Among the systems submitted so far, the strongest results were obtained in the 2025 edition of the task.

The most successful approach so far was submitted in 2025 by the *pjmathematician* team, whose system achieved the highest MAP scores for both English and Portuguese. [1] Their method is notable because it explicitly separates humor recognition from retrieval, combining large language models with dense vector search in a multi-stage architecture.

The approach is built around a two-phase pipeline:

1. Content Analysis and Filtering using LLMs

Every document is processed with a large language model from the Qwen3 family (Qwen3-14B or Qwen3-32B) using a unified prompt that identifies whether the text is a joke and produces a short explanation. This yields an `isJoke` flag and an associated explanation for each item, which are later used both to filter the corpus and to augment the document representations.

2. Dense Retrieval

   In this step a dense retrieval using either Qwen3-4B or Qwen3-8B is applied in order to retrieve based on the queries. The retrieval module itself remains fixed, but the authors evaluate several configurations by varying whether:

   - the corpus is filtered using the `isJoke` labels,
   - documents are indexed in their original form or in an augmented form that includes the explanation, and
   - the query is encoded with a generic retrieval prompt (“Given a web search query, retrieve relevant passages that answer the query”) or a humor-oriented prompt (“Given a query, retrieve relevant jokes related to the query”).

Enhancing Humor-Aware Information Retrieval with Relevance-Aware Classification

A second competitive approach for Task 1 was introduced by Chen, Zhong and Kong in the CLEF 2025 edition [2]. Unlike the multi-stage LLM-driven architecture of the *Qwen LLM* system, this method adopts a more modular and traditional IR + classification design.

The key idea is to decouple query relevance from humor detection, addressing each requirement through its own dedicated model.

The method is organized into two main components:

1. Information Retrieval Step

This module is designed solely to retrieve texts that are topically related to a given query.

More exactly, all documents are embedded using the multilingual *paraphrase-multilingual-mpnet-base-v2* encoder. For each query, cosine similarity is computed between the query vector and every document vector, and all documents that score above a threshold (which is 0) are collected as candidate relevant texts.

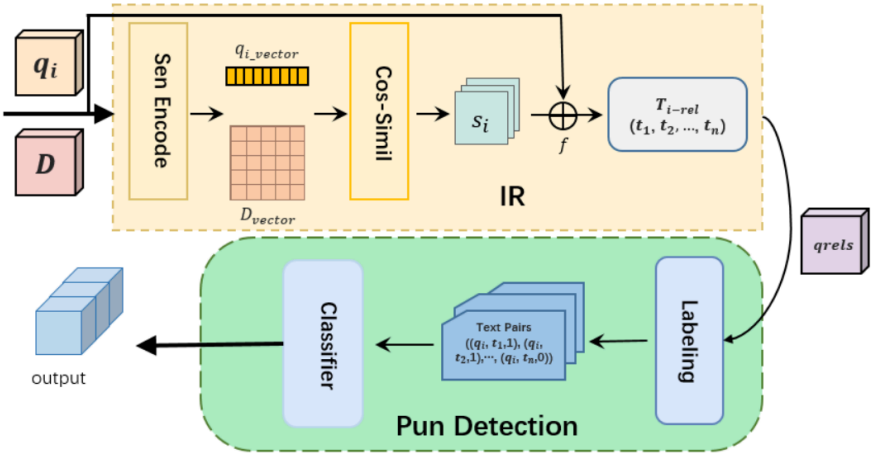
2. Pun Detection Step

In the second stage, humor detection is framed as a binary classification problem, where the system learns to determine whether a given query–document pair should be considered a humorous and relevant match. To train the classifier, the authors construct a supervised dataset in which each document retrieved during the first stage is paired with its corresponding query. These query–document pairs are then assigned labels based on the official *qrel* annotations: pairs that correspond to humorous texts relevant to the query are marked as positive examples, while all remaining pairs are treated as negative examples.

Because the retrieval step intentionally gathers a broad set of documents, the number of negative pairs greatly exceeds the number of positive ones. This creates a strong class imbalance, so the authors apply controlled negative sampling, selecting only a subset of negative examples, to ensure that the classifier is trained on a more balanced and informative dataset.

After assembling the balanced training set, the *xlm-roberta-base* model is fine-tuned as a binary classifier using **cross-entropy loss**. During training, the authors track the classifier's **F1 score** and retain the checkpoint that achieves the highest value. This trained classifier is then applied to the documents obtained from the retrieval stage, determining for each query-document pair whether it contains relevant wordplay.

Those steps are summarized visually in the figure below, which illustrates the full end-to-end flow of the system:



Evaluation Metrics

Task 1 is evaluated using the official CLEF/TREC ranking metrics, which measure both the quality and ordering of retrieved humorous documents. The main metrics are:

- **P@5, P@10, P@15:** precision at cutoff positions, indicating the proportion of relevant items among the top retrieved results.
  - General formula for our problem:
    - $P@K = \frac{\#\{docs\ with\ qrel{=}1\ \ in\ top\ K\ retrieved\}}{K}$
- **MAP (Mean Average Precision):** the mean of the average precision values computed over all queries.
  - In order to present this metric, it is first necessary to introduce the following notion:
    - Average Precision: $AP = \frac{1}{R}\sum_{K=1}^{N} P@K \cdot qrel(K)$, where:
      - $N = \#\{retrieved\ docs\}$
      - $R = \#\{docs\ with\ qrel = 1\ expected\}$
      - $qrel(K) = 1\ if\ the\ document\ at\ rank\ K\ has\ qrel = 1$
  - General formula:
    - $MAP = \frac{1}{Q}\sum_{q=1}^{Q} AP_q$, where:
      - $Q = \#\{queries\}$
      - $AP_q = Average\ Precision\ for\ query\ q$
- **nDCG (Normalized Discounted Cumulative Gain):** evaluates ranking quality by rewarding relevant documents more when they appear higher in the ranking.
  - In order to present this metric, it is first necessary to introduce the following metrics:
    - Discounted Cumulative Gain at K: $DCG@K = \sum_{i=1}^{K} \frac{qrel(i)}{log_2(i+1)}$

        - Ideal Discounted Cumulative Gain at K:

$$IDCG@K = \sum_{i=1}^{min(R,K)} \frac{1}{log_2(i+1)}$$

    - General formula:
        - $nDCG@K = \frac{DCG@K}{IDCG@K}$
- **MRR (Mean Reciprocal Rank):** the average reciprocal rank of the first relevant document retrieved for each query.
    - General formula:
        - $MRR = \frac{1}{Q} \cdot \sum_{q=1}^{Q} \frac{1}{rank_q}$

Results

Across both English and Portuguese datasets, the *Qwen LLM family* system remains the strongest overall, especially on MAP and MRR. The *other* method performs consistently but stays behind on most metrics, with a few exceptions in Portuguese.

All reported scores in the tables are expressed as percentages.

Train (English)

Method	MAP	GMAP	P@R	MRR	P@5	P@10	P@100	P@1000	NDCG@
Qwen LLM family*	48.10	35.02	47.15	78.47	66.67	53.33	18.50	3.29	67.55
Retrieval-then-classification.	59.28	56.10	61.41	69.72	60.00	52.50	23.75	5.35	60.87

Test (English)

Method	MAP	GMAP	P@R	MRR	P@5	P@10	P@100	P@1000	NDCG@
Qwen LLM family*	35.01	24.65	38.68	79.04	54.88	40.63	9.05	1.45	60.80
Retrieval-then-classification.	16.21	NaN	20.59	64.93	35.92	24.47	3.92	0.39	41.34

*Note: for this result, Qwen3-8B was used as the retriever, Qwen3-14B as the humor filter, and Qwen3-32B as the explanation generator

Evaluation of multiple models for pun detection and where they fail

The study “Pun Unintended: LLMs and the Illusion of Humor Understanding” investigates whether *Large Language Models* actually *understand* wordplay or merely exploit patterns to detect them. Puns rely on **polysemy** (words with multiple meanings) or **phonetic similarity** (sound-alike words) to create humor [3].

There were multiple methods of describing how the model should find a pun, including analyzing the *word/s pouns,* the *alternative word,* the whole pair and even their senses. But for our task we will focus on the simplest method: respond with yes, if a given document is/contains wordplay, or no, if it doesn’t.

The study tested 7 LLMs, and they found out that models don`t truly understand puns, however that's less important for us, we can just focus on what model/s got the best results for our task, which is answering a simple question: does the input document contain wordplay? The evaluation criterion was F1-score, which balances Precision and Recall:

GPT-4o seems to be the best model, obtaining a score of 77% on the JOKER dataset, given the 0s problem (simple yes or no answer).

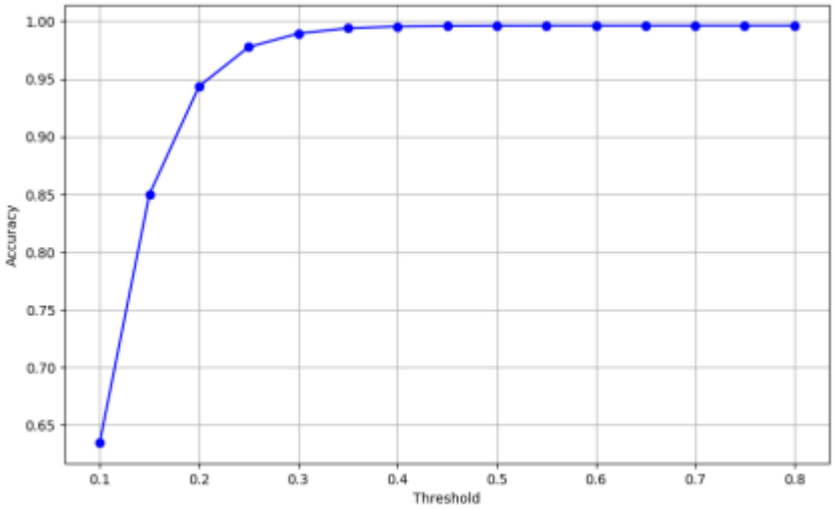
Documents retrieval

According to a paper that focuses specifically on the first task of the JOKER problem, to find all documents relevant to a query a good solution was extending the query with its synonyms and compared similarity not only with an initial query but also with the synonyms. The extension contained up to three total expressions [4]. For example the query {horse} could become {horse, sawhorse, gymnastic horse} while {Tom} would remain the same.

The second step was tokenization, and they compared two different models: bert-base-uncased and all-MiniLM-L6-v2. The latter won in the accuracy test and was kept as the final model for tokenization (0.98 accuracy versus 0.92).

In the end they focused on finding a good similarity score for the accuracy, in order to decide what documents to keep for a certain topic. The chosen threshold is **0.35**, where the F1 score reaches its peak.

M	Prompt	F1-score (%)		
		NAP	JOKER	PunEval
Gemini2.0	0S	73.3 ± 0.0	71.2 ± 0.0	85.7 ± 0.2
	FS	74.9 ± 0.0	74.0 ± 0.1	89.3 ± 0.1
	W	76.2 ± 0.4	74.4 ± 0.1	88.3 ± 0.1
	W+S	**77.9** ± 0.3	74.4 ± 0.0	89.1 ± 0.2
	R+FS	70.9 ± 0.6	71.1 ± 0.2	82.7 ± 0.3
	R+W	76.6 ± 0.0	74.4 ± 0.0	**90.6** ± 0.1
	R+W+S	75.4 ± 0.2	**74.6** ± 0.1	89.5 ± 0.1
GPT-4o	0S	78.4 ± 0.1	77.2 ± 0.0	89.7 ± 0.1
	FS	81.0 ± 0.7	79.7 ± 0.1	92.8 ± 0.2
	W	**86.9** ± 0.8	**83.3** ± 0.4	87.3 ± 0.9
	W+S	85.0 ± 0.8	82.6 ± 0.1	88.9 ± 0.9
	R+FS	82.9 ± 0.0	80.5 ± 0.3	92.3 ± 0.1
	R+W	84.0 ± 1.1	81.1 ± 0.2	92.6 ± 0.4
	R+W+S	82.9 ± 0.8	81.3 ± 0.3	**93.0** ± 0.3
Llama3.3 (70B)	0S	79.6 ± 1.5	77.1 ± 0.8	84.0 ± 0.4
	FS	81.0 ± 0.7	77.6 ± 0.1	84.4 ± 0.4
	W	83.4 ± 0.9	**79.2** ± 0.2	**87.2** ± 0.2
	W+S	**83.6** ± 0.9	77.9 ± 0.6	86.4 ± 0.3
	R+FS	79.2 ± 0.2	75.2 ± 0.1	83.8 ± 0.2
	R+W	78.1 ± 0.8	76.4 ± 0.1	85.1 ± 0.5
	R+W+S	78.3 ± 1.0	76.8 ± 0.2	84.5 ± 0.3
Mistral3 (24B)	0S	**75.0** ± 0.5	**75.3** ± 0.2	80.8 ± 0.2
	FS	69.5 ± 2.2	66.2 ± 1.2	74.6 ± 1.9
	W	69.0 ± 1.7	69.3 ± 0.4	78.4 ± 0.5
	W+S	68.5 ± 0.9	68.5 ± 0.4	74.9 ± 0.9
	R+FS	73.6 ± 1.1	70.9 ± 0.1	82.4 ± 1.3
	R+W	70.7 ± 0.3	69.0 ± 0.4	84.2 ± 0.4
	R+W+S	70.9 ± 0.2	70.1 ± 0.3	**84.5** ± 0.2
Qwen2.5 (72B)	0S	71.3 ± 0.4	73.6 ± 0.2	83.6 ± 0.1
	FS	78.6 ± 0.4	76.4 ± 0.7	87.2 ± 0.2
	W	**81.1** ± 0.0	77.1 ± 0.4	86.8 ± 0.4
	W+S	80.7 ± 0.6	76.8 ± 0.8	87.3 ± 0.1
	R+FS	80.2 ± 0.7	**77.5** ± 0.6	88.5 ± 0.6
	R+W	77.2 ± 0.2	75.0 ± 0.1	**89.4** ± 0.5
	R+W+S	77.0 ± 1.1	75.8 ± 0.6	88.7 ± 0.1
R1-D (70B)	0S	74.8 ± 0.5	75.3 ± 0.6	86.7 ± 0.3
	FS	76.3 ± 2.7	76.2 ± 0.5	87.0 ± 0.3
	W	**79.8** ± 1.4	78.1 ± 0.9	86.8 ± 0.4
	W+S	78.7 ± 1.2	**78.7** ± 0.6	**88.3** ± 0.6
R1 (671B)	0S	74.2 ± 0.4	75.5 ± 0.5	88.5 ± 0.2
	FS	76.6 ± 0.5	78.4 ± 0.4	90.8 ± 0.2
	W	77.7 ± 0.4	77.7 ± 0.1	90.9 ± 0.1
	W+S	**79.1** ± 0.8	**79.2** ± 0.0	**91.3** ± 0.4
RoBERTa-L		64.0 ± 0.1	65.7 ± 0.1	92.5 ± 0.3



Exploratory Data Analysis (EDA)

The objective of this Exploratory Data Analysis (EDA) was to assess the quality, structure, and underlying statistical properties of the text corpus dataset. The analysis focused on handling data quality issues (missing values, outliers) and deriving statistical insights using feature engineering.

1. Data Loading and Quality Assessment

The initial phase confirmed the dataset contained 77,658 records, consisting of ***docid*** (identifier) and ***text*** (content).

Handling Missing Values

A check for data completeness revealed a minimal number of missing values.

- **Initial Shape:** (77,658, 2)
- **Final Shape (After Cleaning):** (77,656, 2)

The missing data was confined to the ***text*** column (2 records), as shown in the output below. Due to the negligible quantity (<0.003%), these records were dropped to ensure the integrity of the subsequent feature engineering steps.

	Missing Count
docid	0

text	2

2. Feature Engineering and Data Structure

To enable quantitative statistical analysis, three numerical features were engineered from the raw text:

1. ***`text_length`***: Total number of characters.
2. ***`word_count`***: Total number of words.
3. ***`unique_word_count`***: Total number of distinct words.

The structure of the data following this transformation is in the next table.

text	**text_length**	**word_count**	**unique_word_count**
He has a green body, no visible nose, and lives in a trash can.	63	14	13
On the software side, a port is a numerical identifier for a specific type of service or application listening on a network or device.	134	24	20
A garbage man going through the trash says: "I'm really messed up today."	73	13	13
The term "run out" can have several meanings depending on the context...	172	29	28

According to the Centers for Disease Control and Prevention dietary guidelines...	333	51	43

3. Univariate Analysis and Outlier Detection

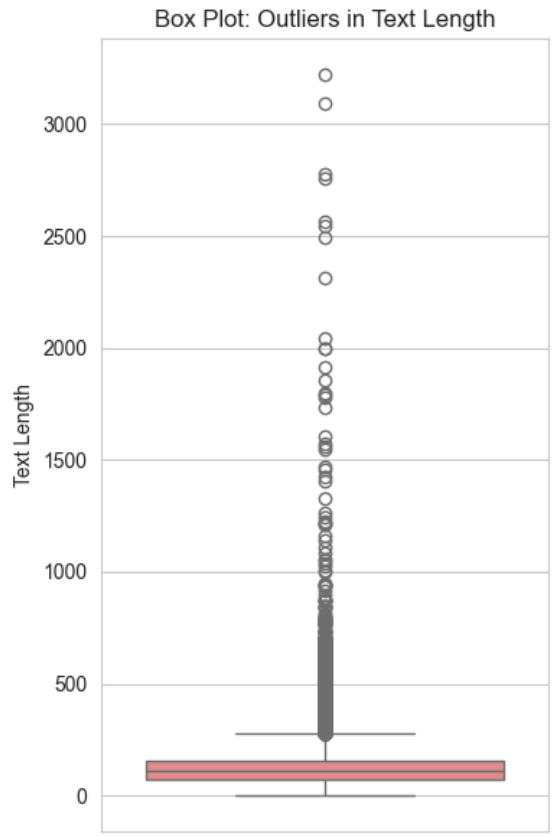
Central Tendency, Dispersion, and Outliers

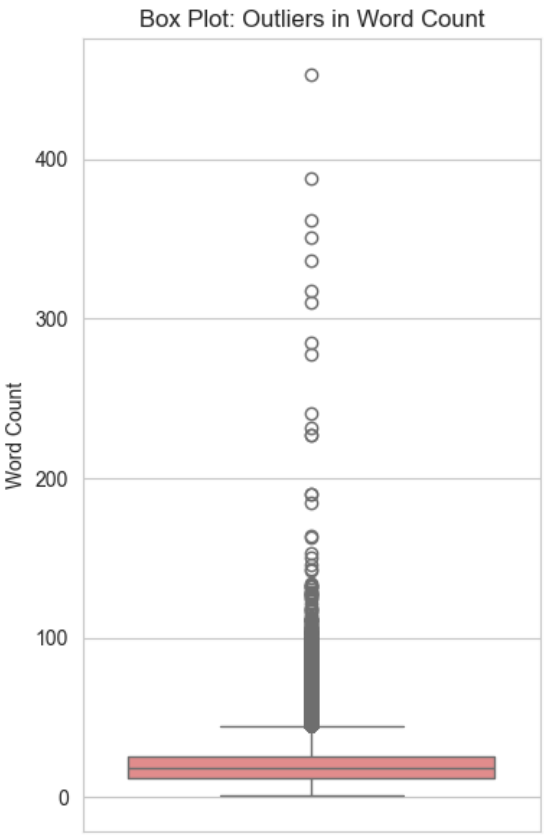
Variable	mean	std	min	Q1	Median (Q2)	Q3	max	P95
text_length	125.494	84.470	1	74	110	156	3219	258
word_count	20.371	12.252	1	12	18	25	453	41
unique_word_count	18.244	9.247	1	12	17	23	257	34

Median vs. Mean: For ***`text_length`***, the mean (125.49) is significantly higher than the median (110), immediately suggesting a heavily right-skewed distribution.

Outlier Severity: The existence of extreme outliers is confirmed by the large gap between the 95th percentile (P95, 258) and the maximum value (max, 3219) for ***`text_length`***. These outliers represent a small fraction of exceptionally long documents.

The presence of these numerous extreme data points is visually evident in the **Box Plot: Outliers in Text Length** and **Box Plot: Outliers in Word Count** . While treated as outliers statistically, these points were retained as they represent valid, albeit rare, documents.

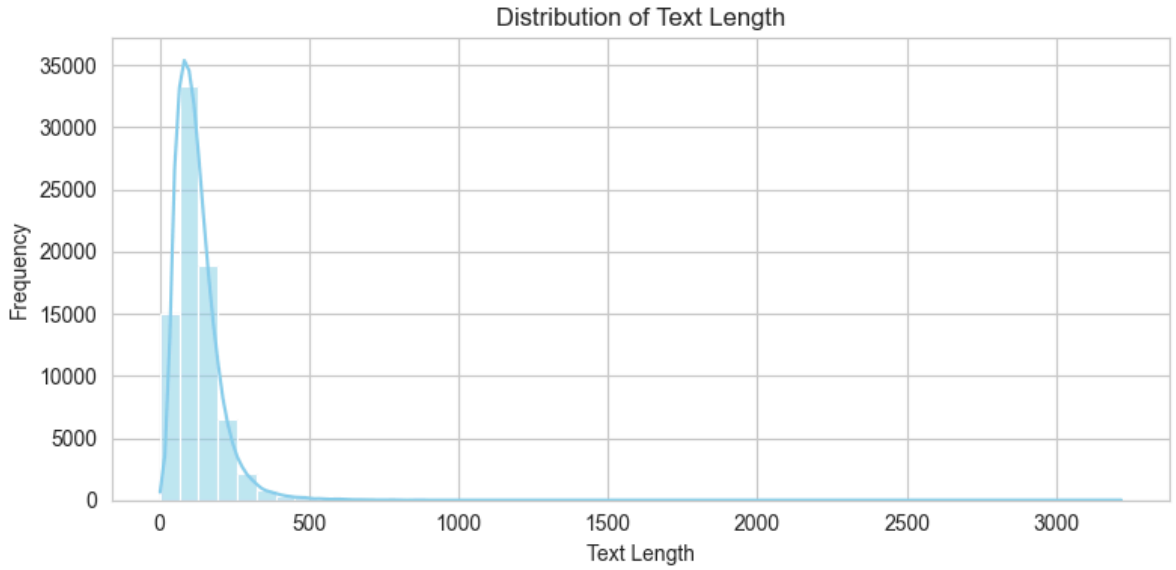


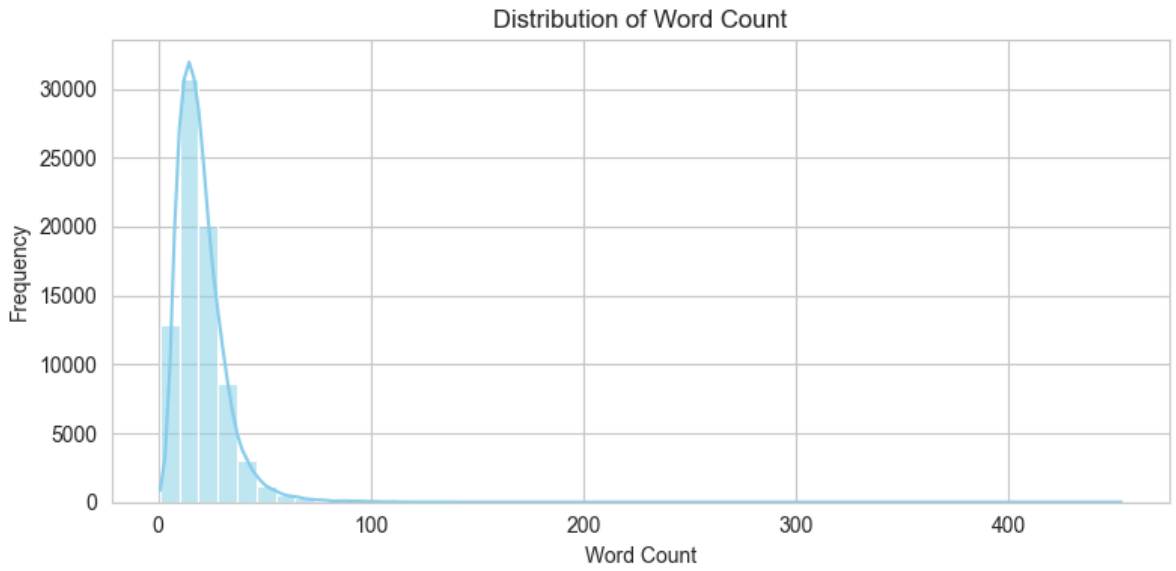


Distribution Shape
The distribution shape was quantified using the Skewness metric.

Variable	Skewness
text_length	6.53634
word_count	4.38309
unique_word_count	2.32777

Interpretation: All features exhibit a high degree of **positive skewness** (values greater than 2.3). This confirms that the majority of documents are short, with the distribution tail pulled toward higher values by the long documents.
This is clearly illustrated in the **Distribution of Text Length** and **Distribution of Word Count** histograms, which show the data concentrated near zero.





4. Multivariate Analysis: Relationship Between Variables

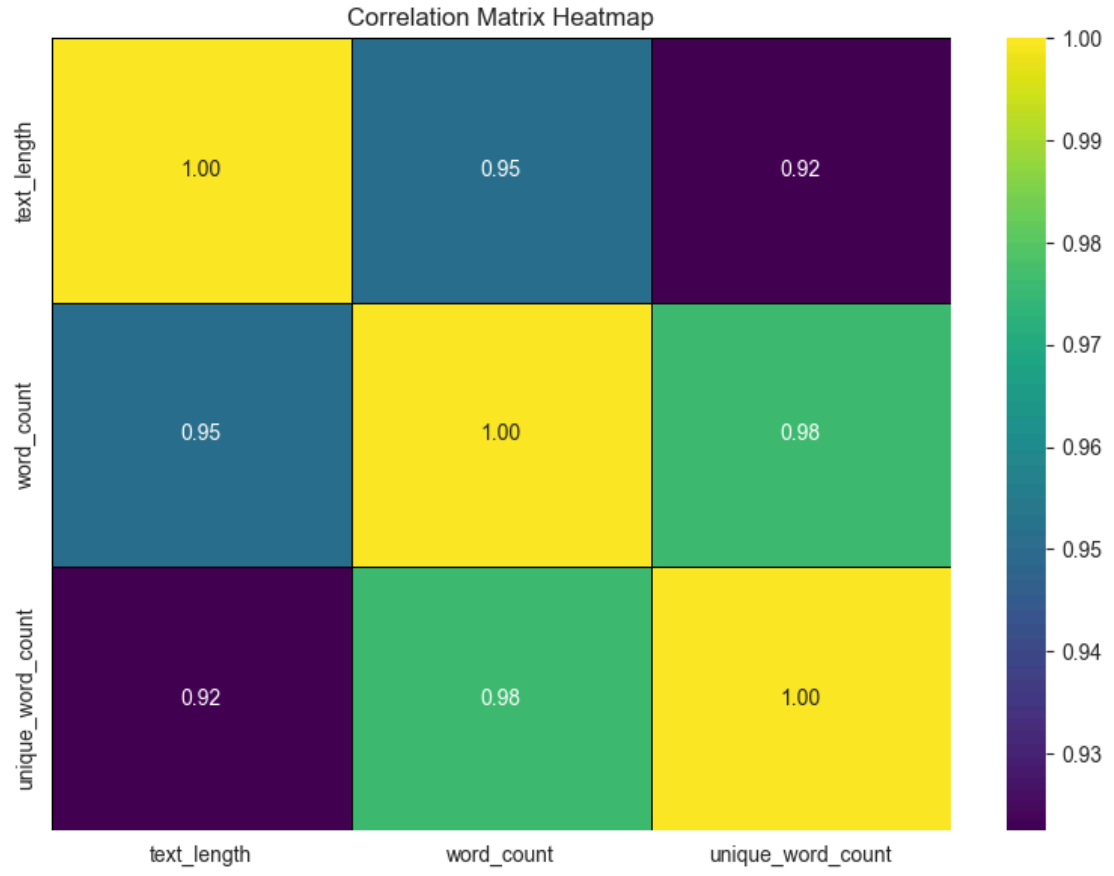
The relationship between the derived features was assessed using the Pearson correlation coefficient.

	text_length	**word_count**	**unique_word_count**
text_length	1	0.951654	0.922474

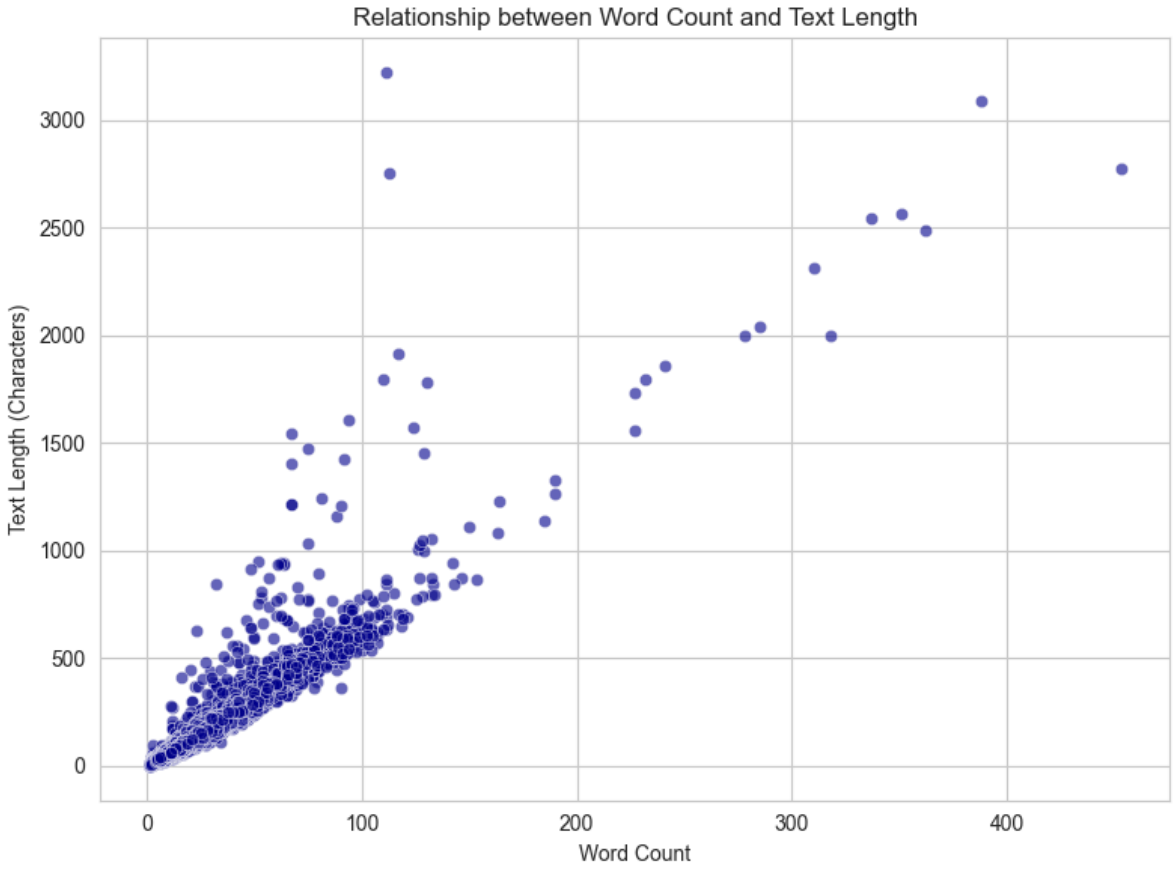
word_count	0.951654	1	0.976101
unique_word_count	0.922474	0.976101	1

All pairs show a **very strong positive correlation** (r > 0.9). This high multicollinearity is expected in text data, as intrinsic length measures are highly related. For instance, the correlation between ***word_count*** and ***unique_word_count*** is 0.976, indicating that longer documents are highly likely to introduce new vocabulary.

Visualization of Relationships
Heatmap: The **Correlation Matrix Heatmap** provides a clear visual confirmation of these strong positive correlations.



Scatter Plot: The **Relationship between Word Count and Text Length** scatter plot confirms the near-perfect linear relationship between these two features.



Conclusion

The analysis demonstrated that the textual data is highly **skewed (most documents are short)** and contains **extreme outliers (a few very long documents)**. Additionally, the measures of length and word count are **strongly related** (multicollinear).

First Implementation Attempt

This implementation follows the relevance-aware classification framework proposed by Chen et al., with a two-stage pipeline consisting of a semantic retrieval module and a pun detection module.

Going into further detail, the most important stages of our implementation are presented below.

Semantic Retrieval Stage

Semantic retrieval is performed using the pre-trained *paraphrase-multilingual-mpnet-base-v2* model. All document texts are encoded once into dense vector representations, computed in batches of 128 documents. The resulting embeddings are L2-normalized and stored for reuse across all queries.

For each query, a dense query embedding is generated using the same model and L2-normalized. Cosine similarity between the query embedding and all document embeddings is computed via dot product. A fixed similarity threshold of 0.0 is applied, and only documents exceeding this threshold are retained as candidates for subsequent stages.

Construction of Training Pairs

To train the pun detection model, the training data is constructed from the documents returned by the semantic retrieval stage. For each query, only the retrieved documents are considered.

The retrieved documents are then labeled using the available relevance judgments (*qrels*). Documents that appear in the *qrels* for a given query are treated as positive training examples. Retrieved documents that do not appear in the *qrels* are treated as negative examples and are considered retrieved but unjudged.

To address class imbalance, negative examples are randomly sampled on a per-query basis such that their number is 50% higher than the number of positive examples. Each training example consists of a query text and a document text paired with a binary label, where label 1 indicates a relevant humorous document and label 0 indicates a negative example.

Pun Detection Training

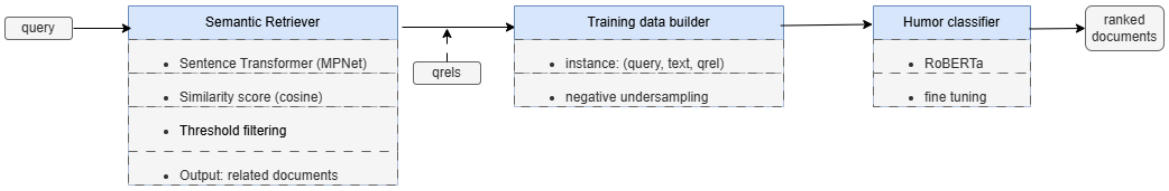
The pun detection step is implemented as a binary classification task over query-document pairs. The classifier is based on *xlm-roberta-base* with a two-class output layer. For each training example, the query text and the document text are provided together as input and tokenized using the corresponding pre-trained tokenizer.

The model is fine-tuned for three epochs using cross-entropy loss and optimized with the AdamW optimizer, with a learning rate of 2e-5. During training, the F1 score computed on the training data is used only for model selection. The model checkpoint with the highest F1 score is saved and used for inference.

Inference and Ranking

After training, the same semantic retrieval procedure and similarity threshold are applied at inference time. For each query, the retrieved documents are paired with the query and passed to the trained pun detection classifier.

For each query-document pair, the classifier outputs the probability of the positive class. Inference is performed in batches of 128 query-document pairs. These probabilities are used directly as ranking scores. For each query, documents are sorted in descending order of their scores to produce the final ranked list.



Experimental results

The first implementation was evaluated under multiple training configurations to analyze the effect of hyperparameter variations on retrieval performance. Starting from an initial setup with a low learning rate and short input sequences, we progressively increased the number of training epochs, learning rate, batch size, and maximum sequence length. The experiments show moderate fluctuations across MAP, nDCG, and recall metrics, with no configuration consistently outperforming the others across all measures. In some cases, longer input sequences and extended training improved early-ranking metrics, while in others they led to lower overall precision.

1. 3 epochs, lr=2e-5, max_len=128, batch_size = 128

Metric	**Value**	**Metric**	**Value**
MAP	0.045	**P@5**	0.055
Recip Rank	0.087	**P@10**	0.058
nDCG@5	0.065	**Recall@5**	0.001
nDCG@10	0.059	**Recall@10**	0.001
nDCG@20	0.055	**Recall@20**	0.002

2. 10 epochs, lr=2e-4, max_len=200, batch_size = 512

Metric	**Value**	**Metric**	**Value**

MAP	0.055	**P@5**	0.066
Recip Rank	0.086	**P@10**	0.066
nDCG@5	0.086	**Recall@5**	0.001
nDCG@10	0.059	**Recall@10**	0.003
nDCG@20	0.068	**Recall@20**	0.013

3. 8epochs, lr=2e-4, max_len=400, batch_size = 512

Metric	Value	Metric	Value
MAP	0.055	**P@5**	0.05
Recip Rank	0.114	**P@10**	0.041
nDCG@5	0.053	**Recall@5**	0.001
nDCG@10	0.045	**Recall@10**	0.001
nDCG@20	0.053	**Recall@20**	0.004

Conclusions

While the overall architecture follows established retrieval–classification designs, the experimental results show limited performance across all evaluated configurations.

Varying training hyperparameters led to only minor performance differences, with no setup consistently outperforming the others. In particular, the semantic retrieval stage, which relies on fixed similarity thresholds and generic sentence embeddings, may limit the set of candidate documents passed to the pun detection stage.

Overall, this implementation serves as a baseline that motivated us to explore alternative approaches in order to achieve improved results.

Second implementation

Architecture

Building on the previous framework, we replaced the XLM-RoBERTa classifier with the Mistral-7B-v0.1 model. While the semantic retrieval stage remains the same, the classification pipeline was updated to use a Large Language Model (LLM) approach for identifying relevant puns.

The model architecture uses the MistralForSequenceClassification class, which adds a classification head to the Mistral transformer. To manage the model's size, we used 4-bit quantization and LoRA (Low-Rank Adaptation) for efficient fine-tuning. We defined a specific prompt format where the model evaluates a query-document pair and is trained to map the relevance to specific label IDs. During inference, the model generates a score based on these labels, effectively deciding how well a document fits the query.

For the ranking stage, we introduced the top_k_semantic parameter to control how many candidates are passed from retrieval to the LLM. The Mistral model processes these pairs in batches, outputting a probability for the "relevant" class. These probabilities are then used as the final scores to rank the documents in descending order.

Experimental results

The following results show how performance shifted when changing the top_k_semantic, top_k and batch_size parameters.

top_k: The final number of documents returned to the user in the ranked list after all processing is complete.
top_k_semantic: The initial number of candidate documents retrieved from the semantic search to be passed to the Mistral model for reranking.
batch_size: The number of query-document pairs that the Mistral model processes at once during reranking to maximize speed and efficiency.

1. Variation 1

Parameters: top_k = 50, top_k_semantic = 100, batch_size = 4

Metric	Value	Metric	Value
MAP	0.1045	**P@5**	0.2000
Recip Rank	0.3493	**P@10**	0.2500
nDCG@5	0.2949	**Recall@5**	0.0299
nDCG@10	0.2958	**Recall@10**	0.0726
nDCG@20	0.2998	**Recall@20**	0.1695

2. Variation 2

Parameters: top_k = 100, top_k_semantic = 250, batch_size = 4

Metric	Value	Metric	Value
MAP	0.1130	**P@5**	0.2160
Recip Rank	0.3703	**P@10**	0.2520
nDCG@5	0.3168	**Recall@5**	0.0323
nDCG@10	0.3197	**Recall@10**	0.0731
nDCG@20	0.3235	**Recall@20**	0.1706

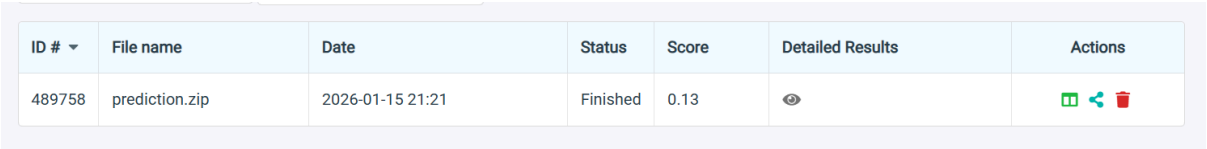
3. Variation 3

Parameters: top_k = 120, top_k_semantic = 300, batch_size = 8

Metric	Value	Metric	Value
MAP	0.1128	**P@5**	0.2160
Recip Rank	0.3703	**P@10**	0.2520
nDCG@5	0.3168	**Recall@5**	0.0323
nDCG@10	0.3197	**Recall@10**	0.0731
nDCG@20	0.3238	**Recall@20**	0.1706

Conclusion

The data indicates that Mistral-7B significantly improves ranking accuracy by identifying relevant puns initially missed by semantic retrieval. Increasing top_k_semantic from 50 to 100 was the most impactful change, yielding an 8.1% boost in MAP and proving that a larger candidate pool allows the LLM to "rescue" relevant documents. However, results plateaued at 120, suggesting a saturation point where extra candidates introduce noise. Meanwhile, doubling the batch_size proved vital for efficiency, maintaining high throughput without compromising precision. Ultimately, the model's ability to refine a fixed top_k output set demonstrates that the LLM-based reranker is far more effective at filtering high-quality humorous content than the baseline semantic search alone.



ID #	File name	Date	Status	Score	Detailed Results	Actions
489758	prediction.zip	2026-01-15 21:21	Finished	0.13		

Prompt Engineering

Model

A prompt engineering method was proposed for solving the problem. By prompt engineering, we refer to focusing on designing instructions/prompts that we pass to the model, and we base our solution on the answers that we are given. There is no training here, so the goal is to use a large, pre-trained, language model that can understand the meaning of the sentences passed to it (important for word play detection), because there is only the inference that matters.

The chosen model is *Mistral-7B-Instruct-v0.3*, which is an instruct fine-tuned version of the Mistral-7B-v0.3. This model understands instructions well and responds clearly, a must when everything is based on the answers. Also it allows for some function called, being helpful for more complex prompts.

Prompts

Experiments were done using 2 different prompts: a basic one and a more complex one.

- Basic prompt:

“Does the following text: {doc} contain wordplay and is relevant to the topic: {topic}? Respond with YES or NO.”,

Where *doc* is the text/document from the dataset (corpus) and topic is the relevant topic.

- Complex prompt:

```
messages = [
    {
        "role": "system",
        "content": (
            "You are a text classification system."
            "Determine whether the text contains humor and whether it is related to the topic."
            "Return JSON only.")
    },
    {
        "role": "user",
        "content": (
            "Topic: death"
            "Text: \"The Easter story is not a dead issue.\"")
    },
    {
        "role": "assistant",
        "content": "{ \"contains_humor\": true, \"about_topic\": true }"
    },
```

```
    {
        "role": "user",
        "content": (
            "Topic: space"
            "Text: \"Mars is the fourth planet from the Sun and has a thin atmosphere.\"")
    },
    {
        "role": "assistant",
        "content": "{ \"contains_humor\": false, \"about_topic\": true }"
    },
    {
        "role": "user",
        "content": (
            "Topic: tom"
            "Text: \"'You resemble a goat," said Tom satirically.\"")
    },
    {
        "role": "assistant",
        "content": "{ \"contains_humor\": true, \"about_topic\": true }"
    },
    {
        "role": "user",
        "content": (
            "Topic: sugar"
            "Text: \"I need to eat more, I am getting weak.\"")
    },
    {
        "role": "assistant",
        "content": "{ \"contains_humor\": false, \"about_topic\": false }"
    },
    {
        "role": "user",
        "content": user_content
    }
 ],
```

Here 2 different techniques are deployed for prompt engineering: first we use *few shots* method, meaning we provide the model with some examples of how the answer is supposed to look like, based on the questions/input given. This lets the model know what we are expecting from our input and the pattern that we want. Secondly, we apply *chat template* in our examples, in order to simulate a real interaction between a chat bot and a user, or maybe even two real persons, this helps separate instructions from questions and improves consistency. This template is consistent across all runs, the only thing that is different across every iteration, is the ***user_content*** string, which is formed by the topic from the respective run and content of that certain iteration.

Since the last message, before generation, was given by the user, now the model will generate an output playing the role of the assistant, and also respecting the pattern already given as an example.

Experimental results

A series of experiments were run in order to evaluate the method proposed. Since the whole dataset contains **77,656 documents**, evaluation for a ***single topic*** would take approximately ***28 hours*** (around 1.3 seconds per document), so it would have been impossible to run tests on the whole datasets, including all documents and all topics. The method was only evaluated on the first 10,000 documents, and only on 2 topics, chosen from those included in the training topic dataset - *queries_train*. The 2 topics chosen are `{"qid":"8","query":"colors"}` and `{"qid":"19","query":"death"}`.

Also a stable method for providing ranking couldn't be found, since generation of a score had big chances of not being consistent, so the metrics used are simply Precision, Recall, F1 score and time.

Results	``colors`` topic		``death`` topic	
Metric	**Basic Prompt**	**Complex Prompt**	**Basic Prompt**	**Complex Prompt**
Precision	0.01129	0.00652	0.00898	0.00716
Recall	0.5	0.75	1	0.71428
F1	0.02209	0.01293	0.0178	0.01418
Time -seconds- average per iteration	1.349	1.349	1.336	1.343
TP	2	3	7	5
FN	2	1	0	2

Results	``colors`` topic		``death`` topic	
FP	175	457	772	693
Wordplay + topic docs	4		7	

It is important to mention that **tokenization**, for the **basic prompt**, was done on the whole input **every iteration**, while on the **complex one**, the hard-coded part was tokenized only once, at the **beginning of the run**, and only the user_content part was tokenized every iteration, since that was actually changing every iteration. This could be an explanation for the very similar time taken to finish an iteration, for the basic and complex prompt.

All the experiments were run without batching the inputs for the model, so one prompt was fed to the model at a time.

Moving forward

Analyzing the results obtained, for most of the false positives, the requirement ***about_topic*** is presented to be **true** by the model, even if the document doesn't involve that topic at all. The ***contains_humor*** requirement is actually very **consistent and accurate**, since the pretrained model used is good at understanding semantics and the meaning behind the words. After a thoughtful examination of the results, the idea of including, inside the content of the system, as an indication or rule, a precise definition of the topic, based on the validation data, seemed promising. This could solve the problem of the model saying that a topic is present or that a document is related to a certain topic even if that was not the case at all, thus making the complex prompt generate more false positive answers when compared to the basic prompt, and drastically improving its accuracy.

Outside this idea, using targeted **topic-related** few shots **examples**, based on the present topic evaluated, for the complex model, seems like a promising idea, since it also adds a *practical* example outside of the given definition, making the model more aware of what about_topic should really be like.

Second Prompt Engineering

Within this approach, which is similar to the prompt-engineering method described above, the system follows a multi-stage retrieval pipeline in which humor analysis and document retrieval are handled as separate steps, inspired by the approach described in [1]. Humor-related processing is performed offline, as it is computationally expensive; this design

decouples humor recognition from retrieval and allows the retrieval component to be modified or reconfigured without repeating the humor analysis, thereby reducing the overall time required to produce results.

Offline Humor Analysis

In an offline preprocessing stage, all documents in the corpus are analyzed using an instruction-tuned language model (Qwen2.5-3B-Instruct). Each document is classified as humorous or non-humorous, and a short explanation describing the humor is generated when applicable.

This step produces an isJoke label and an associated explanation for each document. The results are stored in a cache file (llm_analysis_cache.json) that can be reused. The analysis is performed once and does not take place at query time.

The following prompt is used for humor detection and explanation generation:

```
"""You are a humor analysis assistant.

Given a text, decide whether it is a joke.

Output ONLY a valid JSON object enclosed in <json></json> tags.
Do not include any text outside the tags.

The JSON must contain exactly two fields:
- isJoke: boolean
- explanation: a short one-sentence explanation.

Text:
"""
```

Corpus Filtering and Text Preparation

The humor labels produced during preprocessing are used to filter the corpus, keeping only documents marked as jokes. For the retained documents, the original text is extended with the explanation generated by the language model when available, resulting in an augmented document representation used for embedding.

Dense Retrieval

Document and query representations are computed using a dense embedding model (Qwen3-Embedding-4B). Document embeddings are generated from the filtered and augmented corpus, while queries are encoded using a task-specific formulation oriented toward joke retrieval. Both document and query embeddings are L2-normalized prior to similarity computation.

Similarity between queries and documents is computed using cosine similarity. For each query, documents are ranked based on their similarity scores, and the top-ranked

(Top-k = 1000)results are selected for output. To ensure score comparability within each ranked list, similarity scores are normalized.

Queries are encoded with a task-specific contextual prefix to bias the embedding space toward joke retrieval: `"Given a query, retrieve relevant jokes related to the query: {query}"`.

Results

ID #	File name	Date	Status	Score	Detailed Results	Actions
489910	submission.zip	2026-01-16 00:31	Finished	0.19		

Conclusion

The results obtained with this second implementation suggest that prompt engineering is a promising direction for humor-aware retrieval. Compared to earlier approaches, the prompt-based setup appears to generalize better on the test data, benefiting from richer semantic signals captured during offline humor analysis and from task-oriented query formulation.

As future work, we plan to further explore this direction by varying the language models and prompt designs, and by incorporating an explicit humor confidence score alongside the generated explanations, which could enable more flexible filtering and ranking strategies.

Bibliography

[1] pjmathematician Team. (2025). A Unified Approach to Humour Retrieval and Translation using the Qwen LLM Family. In Proceedings of the CLEF 2025 Conference. CEUR Workshop Proceedings. https://ceur-ws.org/Vol-4038/paper_230.pdf

[2] Chen, Z., Zhong, Y., & Kong, L. (2025). Enhancing Humor-Aware Information Retrieval with Relevance-Aware Classification. In Proceedings of the CLEF 2025 Conference. CEUR Workshop Proceedings. https://ceur-ws.org/Vol-4038/paper_223.pdf

[3] Mao, Y., Fan, Z., & Liu, Q. (2025). Pun Unintended: LLMs and the Illusion of Humor Understanding. arXiv preprint arXiv:2509.12158. https://arxiv.org/pdf/2509.12158

[4] Kiesel, J., Hagen, M., Potthast, M., & Stein, B. (2024). Humor-Aware Information Retrieval: Task Overview and Dataset. In CLEF 2024 Working Notes. CEUR Workshop Proceedings. https://ceur-ws.org/Vol-3740/paper-173.pdf